JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class LongAccumulator

java.lang.Object
    java.lang.Number
        java.util.concurrent.atomic.LongAccumulator

All Implemented Interfaces:

Serializable

```
public class LongAccumulator
extends Number
implements Serializable
```

One or more variables that together maintain a running `long` value updated using a supplied function. When updates (method `accumulate(long)`) are contended across threads, the set of variables may grow dynamically to reduce contention. Method `get()` (or, equivalently, `longValue()`) returns the current value across the variables maintaining updates.

This class is usually preferable to `AtomicLong` when multiple threads update a common value that is used for purposes such as collecting statistics, not for fine-grained synchronization control. Under low update contention, the two classes have similar characteristics. But under high contention, expected throughput of this class is significantly higher, at the expense of higher space consumption.

The order of accumulation within or across threads is not guaranteed and cannot be depended upon, so this class is only applicable to functions for which the order of accumulation does not matter. The supplied accumulator function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument. For example, to maintain a running maximum value, you could supply `Long::max` along with `Long.MIN_VALUE` as the identity.

Class `LongAdder` provides analogs of the functionality of this class for the common special case of maintaining counts and sums. The call `new LongAdder()` is equivalent to `new LongAccumulator((x, y) -> x + y, 0L`.

This class extends `Number`, but does *not* define methods such as `equals`, `hashCode` and `compareTo` because instances are expected to be mutated, and so are not useful as collection keys.

Since:

`1.8`

See Also:

`Serialized Form`

Constructor Summary

Constructors

Constructor	Descriptio
`LongAccumulator(LongBinaryOperator accumulatorFunction, long identity)`	Creates a new instance using the given accumulat function and identi element.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`void`	`accumulate(long x)`	Updates with the given value.
`double`	`doubleValue()`	Returns the **current value** as a `double` after a widening primitive conversion.
`float`	`floatValue()`	Returns the **current value** as a `float` after a widening primitive conversion.
`long`	`get()`	Returns the current value.

long	**getThenReset**()	Equivalent in effect to **get()** followed by **reset()**.
int	**intValue**()	Returns the **current value** as an int after a narrowing primitive conversion.
long	**longValue**()	Equivalent to **get()**.
void	**reset**()	Resets variables maintaining updates to the identity value.
String	**toString**()	Returns the String representation of the current value.

Methods inherited from class java.lang.Number

byteValue, shortValue

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

LongAccumulator

```
public LongAccumulator(LongBinaryOperator accumulatorFunction,
                       long identity)
```

Creates a new instance using the given accumulator function and identity element.

Parameters:

`accumulatorFunction - a side-effect-free function of two arguments`

`identity - identity (initial value) for the accumulator function`

Method Detail

accumulate

```
public void accumulate(long x)
```

Updates with the given value.

Parameters:

`x - the value`

get

```
public long get()
```

Returns the current value. The returned value is *NOT* an atomic snapshot; invocation in the absence of concurrent updates returns an accurate result, but concurrent updates that occur while the value is being calculated might not be incorporated.

Returns:

`the current value`

reset

```
public void reset()
```

Resets variables maintaining updates to the identity value. This method may be a useful alternative to creating a new updater, but is only effective if there are no concurrent updates. Because this method is intrinsically racy, it should only be used when it is known that no threads are concurrently updating.

getThenReset

```
public long getThenReset()
```

Equivalent in effect to `get()` followed by `reset()`. This method may apply for example during quiescent points between multithreaded computations. If there are updates concurrent with this method, the returned value is *not* guaranteed to be the final value occurring before the reset.

Returns:

`the value before reset`

toString

```
public String toString()
```

Returns the String representation of the current value.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current value

longValue

```
public long longValue()
```

Equivalent to `get()`.

Specified by:

`longValue` in class `Number`

Returns:

the current value

intValue

```
public int intValue()
```

Returns the current value as an `int` after a narrowing primitive conversion.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `int`.

floatValue

```
public float floatValue()
```

Returns the current value as a `float` after a widening primitive conversion.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type float.

doubleValue

```
public double doubleValue()
```

Returns the current value as a `double` after a widening primitive conversion.

Specified by:

`doubleValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type double.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.